

Documento Técnico: Diseño de Flujos de Usuario — Proyecto HeartCoin

Introducción

Este documento describe, pantalla por pantalla, los flujos de usuario efectivamente implementados en la aplicación cliente de HeartCoin para sus tres roles (Personal, Organización, Empresa), desde el primer contacto con la aplicación hasta las operaciones transaccionales centrales del sistema (votación, check-in, canje de HeartCoin). Se documenta el diseño de flujos tal como existe hoy en el código fuente de la aplicación, no una maqueta de referencia: cada transición de pantalla descrita corresponde a una llamada de navegación verificada en el proyecto.

Objetivo

Documentar el recorrido completo de cada rol de usuario dentro de la aplicación, identificando qué pantalla conduce a cuál, qué decisión o acción del usuario dispara cada transición, y confirmar que los flujos transaccionales de mayor relevancia (autenticación, votación, check-in, canje de recompensas) están operativos de principio a fin sobre el backend real.

Alcance

Este documento cubre la totalidad de las pantallas presentes en `lib/screens/` al momento de su redacción, organizadas por el rol al que pertenecen funcionalmente, y el mapa completo de rutas nombradas registradas en `lib/main.dart`. Se señalan explícitamente los casos en que una pantalla existe en el código pero no está enganchada a ningún flujo de navegación activo, y los casos en que una sección de una pantalla está marcada como "Próximamente" y no corresponde a un flujo completo. No cubre la correspondencia de estos flujos con las maquetas de diseño de Figma, por no haberse podido acceder al contenido detallado de esas maquetas al momento de redactar este documento.

Metodología

La información se obtuvo mediante inspección directa del código fuente de cada pantalla y de los servicios que estas invocan, verificando cada transición de navegación (`Navigator.push`, `pushNamed`, `pushReplacement`, `pushAndRemoveUntil`) y cada llamada a servicios de backend (lectura/escritura directa, invocación de funciones RPC, suscripción en tiempo real). Se realizó una búsqueda global

sobre el proyecto para confirmar, en cada caso señalado como excepción, que efectivamente no existe una referencia de navegación adicional no detectada en la primera revisión.

Desarrollo del Análisis

1. Flujo de Entrada: Onboarding y Autenticación

Este flujo es compartido por los tres roles y es el único punto de entrada a la aplicación. La secuencia real de pantallas es la siguiente:

```
OnboardingScreen (3 slides informativos) → HomeScreen (pantalla de bienvenida con imagen de fondo) → "Registrarse" → RegisterScreen → "Iniciar sesión" → LoginScreen
```

OnboardingScreen es la pantalla configurada como `home:` de la aplicación (`lib/main.dart`). Presenta tres slides ("Bienvenid@ a HeartCoin", "Acumula y comparte", "Canjea tus recompensas") mediante un `PageView`, y al llegar al final reemplaza la pila de navegación hacia **HomeScreen** — una pantalla de bienvenida intermedia, distinta de las pantallas "home" por rol descritas más adelante, cuya única función es ofrecer los botones "Registrarse" e "Iniciar sesión".

Registro

RegisterScreen presenta un selector de rol en formato de pestañas (Personal / Organización / Empresa) que determina qué campos adicionales se solicitan: nombre completo y tipo de perfil para Personal; nombre de organización, representante, cargo, tipo de organización y área de impacto para Organización; nombre de empresa, representante, cargo, industria, rango de empleados y objetivo principal para Empresa. País, ciudad, correo y contraseña son comunes a los tres. Al enviar el formulario:

```
RegisterScreen (formulario + selección de rol) → AuthService.registerPersonal / registerOrganization / registerCompany → Supabase Auth signUp (guarda el rol en user_metadata) → insert en personal_profiles / organization_profiles / company_profiles → OriginalAuthService.startEmailVerification() → VerifyEmailScreen (pushAndRemoveUntil)
```

El envío del formulario ejecuta, en secuencia, tres operaciones contra el backend: la creación de la cuenta de autenticación, la inserción del perfil inicial correspondiente al rol elegido, y el disparo de un código de verificación de correo a través de un servicio externo (*Original Auth*), independiente del mecanismo nativo de Supabase Auth.

Distinción importante: la aplicación utiliza dos mecanismos de autenticación relacionados pero independientes entre sí. La **verificación de correo posterior al registro** se resuelve mediante *Original Auth* (un servicio externo con su propio flujo de código y reenvío, en `VerifyEmailScreen`). La **recuperación de contraseña**, en cambio, utiliza el mecanismo nativo de Supabase Auth (`resetPasswordForEmail` y `verifyOTP` con `OtpType.recovery`). Ambos flujos piden un "código de verificación" al usuario, pero no comparten implementación ni backend.

VerifyEmailScreen solicita el código enviado por correo y lo valida contra *Original Auth*. Al verificarse correctamente, navega —reemplazando toda la pila de navegación— a la pantalla principal correspondiente al rol registrado (`/home-personal`, `/home-organizacion` o `/home-empresa`). **La aplicación bloquea el acceso a cualquier otra pantalla hasta completar este paso**, como decisión de producto explícita: no existe una ruta alternativa para omitir la verificación. La pantalla ofrece "Reenviar código" y "Cerrar sesión y salir" (retorno a `LoginScreen`) como únicas acciones alternativas.

Inicio de sesión

```
LoginScreen (correo + contraseña) → AuthService.signIn() → Supabase Auth
signInWithPassword → lectura de rol desde user_metadata → /home-personal | /home-
organizacion | /home-empresa
```

LoginScreen autentica directamente contra Supabase Auth y determina, a partir del rol almacenado en los metadatos del usuario, a cuál de las tres pantallas principales redirigir —sin pasar de nuevo por verificación de correo, dado que esta ya se completó en el registro. Desde esta pantalla también se accede a **ForgotPasswordScreen** mediante el enlace "¿Olvidaste tu contraseña?".

Recuperación de contraseña

```
ForgotPasswordScreen (correo) → Supabase Auth resetPasswordForEmail →
VerifyResetCodeScreen (código de 6 dígitos) → Supabase Auth verifyOTP (type: recovery) →
sesión temporal → ResetPasswordScreen (nueva contraseña + confirmación) → Supabase Auth
updateUser → signOut() → LoginScreen
```

Este flujo cierra deliberadamente la sesión al finalizar, obligando al usuario a iniciar sesión de nuevo con la nueva contraseña —una decisión de seguridad que evita dejar una sesión de recuperación activa una vez completado el cambio.

2. Mapa de Rutas Nombradas

La aplicación registra un subconjunto de sus pantallas como rutas nombradas en `lib/main.dart` ; el resto se alcanza exclusivamente mediante navegación directa (`Navigator.push` sin nombre de ruta), generalmente porque requieren un parámetro específico (un identificador de iniciativa, beneficio, usuario, etc.) que no se presta a una ruta fija.

Ruta	Pantalla	Rol principal
<code>/home-personal</code>	HomePeopleScreen	Personal
<code>/home-organizacion</code>	HomeOrganizationScreen	Organización
<code>/home-empresa</code>	HomeCompanyScreen	Empresa
<code>/search</code>	ExploreScreen	Personal
<code>/wallet</code>	WalletScreen	Personal
<code>/votaciones</code>	VotacionesScreen	Personal
<code>/certificados</code>	CertificadosScreen	Personal
<code>/iniciativas-ahorro</code>	IniciativasAhorroScreen	Personal
<code>/iniciativas-social</code>	IniciativasSocialScreen	Personal
<code>/create-post</code>	CreatePostScreen	Personal
<code>/mis-acciones</code>	MisAccionesScreen	Personal
<code>/profile</code>	ProfileScreen	Común
<code>/settings</code>	SettingsScreen	Común
<code>/mis-iniciativas</code>	OrgIniciativasScreen	Organización
<code>/votaciones-monitor</code>	VotacionesMonitorScreen	Organización
<code>/comunidad</code>	CommunityScreen	Organización
<code>/mis-beneficios</code>	CompanyBeneficiosScreen	Empresa

Nota de precisión: tres pantallas del flujo de Empresa — `CompanyServiciosScreen` , `CreateBeneficioScreen` y `CreateServicioScreen` — no tienen ruta nombrada registrada; se alcanzan exclusivamente mediante navegación directa desde el bottom nav o desde botones de acción dentro de otras pantallas del mismo rol.

3. Flujo del Rol Personal

Home y navegación principal

HomePeopleScreen es un feed en tiempo real de publicaciones (tabla `publicaciones`), filtrado localmente para excluir a los autores que el usuario haya bloqueado. La barra superior incluye un menú lateral (*drawer*) y un ícono de notificaciones con contador de no leídas. Un botón flotante permite crear una publicación nueva. La barra de navegación inferior es consistente en todo el flujo Personal:

```
Bottom nav (Personal): Home | Buscar → /search | [centro] Wallet → /wallet | Tus acciones
→ /mis-acciones | Perfil → /profile
```

El menú lateral (*drawer*) es el punto de acceso a las secciones que no viven en la barra inferior: Mi Billetera (`/wallet`), Votaciones (`/votaciones`), Certificados (`/certificados`), Iniciativas de ahorro (`/iniciativas-ahorro`), Iniciativas social (`/iniciativas-social`), Configuración (`/settings`) y Cerrar sesión.

Cada tarjeta de publicación en el feed permite navegar al perfil del autor (**UserProfileScreen**), a la iniciativa relacionada si la publicación la etiquetó (**IniciativaDetailScreen**), abrir el panel de comentarios, o bloquear al autor mediante un menú contextual con confirmación previa.

Explorar e iniciativas

```
ExploreScreen (/search) → tarjetas de categoría (Voluntariado, Crowdfunding, Social,
Ahorro) → sección "Cerca de ti" (si el usuario habilitó ubicación) → IniciativaCard → tap
→ IniciativaDetailScreen
```

IniciativaDetailScreen se bifurca según la categoría de la iniciativa: si es "Ahorro", delega su cuerpo completo a un componente especializado (**IniciativaAhorroDetailBody**); en cualquier otro caso, muestra pestañas de Detalles / Comentarios / Presupuesto (las dos últimas marcadas como "Próximamente"), un botón para apoyar la iniciativa (registra un voto) y, si corresponde, un botón "Escanear QR para check-in" que abre **CheckinScannerScreen**.

El detalle de una iniciativa de ahorro tiene su propio conjunto de pestañas (General / Proveedor / Actividad) y un sistema de participantes real: el dueño de la iniciativa puede editar la meta de ahorro, invitar participantes (**InviteParticipantScreen**) y aceptar o rechazar solicitudes de unión; un usuario que no es dueño ni participante ve una tarjeta para solicitar unirse. Las secciones "Avances reportados", "Contrato" y la pestaña "Actividad" están explícitamente marcadas como "Próximamente" — no corresponden a un flujo completo todavía, dado que no existe aún un panel de proveedor que alimente esos datos.

Votaciones

VotacionesScreen (/votaciones) → buscador + filtros (Todas / Por tiempo / +HC) → _VotacionCard → botón "Votar" (directo) | tap en tarjeta → VotacionDetailScreen → VotacionDetailScreen: pestañas Detalles / Comentarios / Presupuesto → botón "Votar por esta iniciativa"

El botón "Votar" está disponible tanto en la tarjeta de la lista como en el detalle, y se deshabilita si el usuario ya emitió su voto para esa votación específica. Los comentarios de la pestaña correspondiente se leen y escriben en tiempo real.

Billetera (Wallet)

WalletScreen (/wallet) es la pantalla de mayor densidad funcional del flujo Personal, organizada en cuatro pestañas:

Pestaña	Contenido	Navegación de salida
Wallet	Historial de transacciones en tiempo real (voto, check-in, canje, cashback, ajuste)	Banner hacia /iniciativas-ahorro
Beneficios	Buscador, categorías, secciones Destacados / Para ti, mapa de beneficios cercanos	Tap en tarjeta → BeneficioDetailScreen
Servicios	Mismo patrón que Beneficios, con categorías propias (Consultoría, Tecnología, Diseño, Legal, Marketing, Capacitación, Otro)	Tap en tarjeta → ServicioDetailScreen
Puntos	Ranking de usuarios por número de check-ins realizados	Sin navegación de salida

El balance de HeartCoin se muestra en un encabezado visible únicamente en la pestaña Wallet, actualizado en tiempo real, junto a un acceso directo al escáner de canje (**BeneficioScannerScreen**).

Canje de HeartCoin

BeneficioDetailScreen / ServicioDetailScreen / Wallet (icono de escaneo) → BeneficioScannerScreen → intenta resolver el código como beneficio → si no corresponde, intenta resolverlo como servicio → confirmación visual del canje

El escáner implementa una resolución automática de tipo: el usuario no indica de antemano si está escaneando un beneficio o un servicio — la pantalla intenta primero interpretar el código como beneficio y, si no corresponde, como servicio, antes de reportar que el código no es válido.

Resto del flujo Personal

Completan el flujo Personal las siguientes pantallas, cada una con su propósito específico:

- **MisAccionesScreen** (`/mis-acciones`): estadísticas acumuladas (HeartCoins, votos, check-ins) y dos vistas — Participaciones (historial de acciones con su efecto en HC) y Guardado (iniciativas y votaciones marcadas para revisar después, con navegación directa a su detalle).
- **ProfileScreen** (`/profile`) y **UserProfileScreen** (perfil de otro usuario): estadísticas de publicaciones, seguidores y seguidos, y grilla de publicaciones propias; el perfil de otro usuario añade un botón de seguir y un menú de bloqueo.
- **CreatePostScreen** (`/create-post`): publicación con texto obligatorio, relación opcional con una iniciativa (mediante un buscador dedicado, **SelectIniciativaScreen**), adjuntos de imagen o documento, y ubicación opcional.
- **NotificationsScreen**: lista en tiempo real de notificaciones, con acceso directo al panel de comentarios de la publicación relacionada cuando aplica.
- **SettingsScreen** (`/settings`): punto de acceso a Editar perfil, Cambiar contraseña, Notificaciones, Privacidad, Acerca de, Usuarios bloqueados, preferencia de ubicación, y la "zona de riesgo" con Eliminar cuenta.
- **CertificadosScreen** (`/certificados`): certificados generados a partir de check-ins confirmados, con vista de detalle tipo diploma.

Eliminación de cuenta

```
SettingsScreen → "Eliminar cuenta" → DeleteAccountScreen → advertencia + checklist de lo
que se borra → casilla de confirmación obligatoria → AccountService.deleteAccount() →
signOut() → retorno a pantalla de inicio
```

La casilla de confirmación es obligatoria antes de habilitar el botón de acción — el usuario no puede eliminar su cuenta por error de un solo toque. La pantalla enumera explícitamente lo que se elimina: perfil y foto, publicaciones, comentarios y likes, balance e historial de HeartCoin, votos, check-ins y certificados, elementos guardados y relaciones de seguimiento.

4. Flujo del Rol Organización

```
HomeOrganizationScreen (/home-organizacion) → tarjeta "Nueva iniciativa" →
CreateOrgIniciativaScreen → resumen de actividad + gráfica de votos por día Drawer:
Monitor de votaciones (/votaciones-monitor) | Comunidad (/comunidad) | Configuración |
Cerrar sesión Bottom nav: Home | Iniciativas (/mis-iniciativas) | [2 íconos sin acción
todavía]
```

HomeOrganizationScreen resume la actividad de la organización (iniciativas activas, votos recibidos, personas alcanzadas) y ofrece acceso directo a la creación de una nueva iniciativa. Dos íconos de la barra de navegación inferior (verificación y grupos) están presentes visualmente pero no tienen acción asignada todavía.

Gestión de iniciativas

```
OrgIniciativasScreen (/mis-iniciativas) → filtro de estado (Todas / Borrador / Votación) →
FAB "+" → CreateOrgIniciativaScreen → por cada iniciativa: "Ver" → IniciativaDetailScreen
"Editar" → EditOrgIniciativaScreen "Mostrar QR" → IniciativaQrScreen
```

CreateOrgIniciativaScreen es un asistente de cuatro pasos: (1) tipo de iniciativa, título, descripción y fecha del evento; (2) actividades, requisitos y cantidad máxima de participantes; (3) método de verificación del check-in (QR, manual o foto), evidencia requerida y ventana horaria de verificación; (4) previsualización y duración de la votación, tras lo cual la iniciativa se crea directamente con estado *"votacion"*.

Nota de precisión: aunque la pantalla de listado ofrece un filtro por estado "Borrador", el asistente de creación no produce iniciativas en ese estado — toda iniciativa nueva entra directamente a fase de votación. El estado "Borrador" existe en el modelo de datos y puede alcanzarse mediante **EditOrgIniciativaScreen**, pero no es el punto de partida del flujo de creación actual.

El código QR generado por **IniciativaQrScreen** corresponde al identificador de la propia iniciativa, y es el mismo valor que **CheckinScannerScreen** (del lado Personal) valida al momento del check-in — confirmando que ambas pantallas, aunque pertenecen a roles distintos, operan sobre el mismo mecanismo de identificación.

Monitor de votaciones

```
VotacionesMonitorScreen (/votaciones-monitor) → selector desplegable de iniciativa (por
defecto, la más reciente) → gráfica de barras: votos por día, últimos 7 días
```

Al seleccionar una iniciativa en el desplegable, la pantalla consulta los votos registrados para esa iniciativa y los agrupa por día para los últimos siete días, mostrando el resultado como una gráfica de barras verticales.

Hallazgo de código: existe una pantalla adicional, **VotacionMonitorDetailScreen** (tasa de aprobación, participación y evolución acumulada de votos en gráfica de línea), completamente

implementada en el código fuente, pero **no está referenciada por ninguna llamada de navegación en todo el proyecto** — se confirmó mediante búsqueda global que ninguna pantalla la invoca. No debe considerarse parte del flujo activo del Monitor de votaciones hasta que se le agregue un punto de entrada (por ejemplo, un tap sobre la gráfica de barras).

Comunidad

CommunityScreen (/comunidad), accesible desde el menú lateral de Organización, lista a los usuarios que han votado o apoyado iniciativas, con buscador por nombre y conteo de participaciones por usuario. Las tarjetas de esta lista no navegan a ningún perfil de detalle — es una vista de consulta, no un punto de entrada a otras pantallas.

5. Flujo del Rol Empresa

HomeCompanyScreen (/home-empresa) → tarjeta "Nuevo beneficio" → CreateBeneficioScreen → resumen de actividad (beneficios activos, por vencer, canjes, HC cobrados) → "Ver todos mis beneficios" → CompanyBeneficiosScreen Drawer: Mis beneficios (/mis-beneficios) | Configuración | Cerrar sesión Bottom nav: Home | Beneficios (/mis-beneficios) | Servicios (push directo, sin ruta nombrada) | [ícono sin acción]

Gestión de beneficios y servicios

CompanyBeneficiosScreen (/mis-beneficios) → FAB "+" → CreateBeneficioScreen → por cada beneficio: tap → BeneficioDetailScreen Activar/Desactivar (directo) "QR" → BeneficioQrScreen CompanyServiciosScreen (push directo desde bottom nav) → FAB "+" → CreateServicioScreen → por cada servicio: tap → ServicioDetailScreen Activar/Desactivar (directo) "QR" → ServicioQrScreen

CreateBeneficioScreen solicita tipo de beneficio (descuento, cashback, beca u otro — determinando si el campo de HeartCoin se interpreta como costo o como recompensa), título, descripción, términos, límite de canjes, fecha de vencimiento, ubicación opcional y un interruptor para marcarlo como "destacado". **CreateServicioScreen** sigue el mismo patrón con categorías propias del ámbito de servicios (Consultoría, Tecnología, Diseño, Legal, Marketing, Capacitación, Otro).

El código QR mostrado en **BeneficioQrScreen** y **ServicioQrScreen** corresponde al identificador del beneficio o servicio respectivo, y es el mismo valor que **BeneficioScannerScreen** (del lado Personal) resuelve automáticamente al momento del canje.

6. Mecanismos de Backend Detrás de los Flujos Transaccionales

Los tres flujos transaccionales centrales del sistema —canje, check-in y votación— no comparten el mismo nivel de validación server-side, y esta diferencia es relevante para entender la robustez real de cada uno:

Canje de HeartCoin (beneficios y servicios)

El cliente **no realiza insert ni update directo** para efectuar un canje. En su lugar, invoca las funciones de Postgres `redeem_beneficio` y `redeem_servicio` mediante RPC, pasando únicamente el identificador del beneficio o servicio. Toda la lógica de negocio —verificación de saldo suficiente, límite de canjes no excedido, estado activo, ausencia de canje previo, descuento o abono de HeartCoin y registro de la transacción— ocurre dentro de la función del servidor. El cliente solo interpreta el resultado: éxito, o uno de los códigos de error tipados (`ya_canjeado`, `limite_alcanzado`, `hc_insuficiente`, `beneficio_inactivo / servicio_inactivo`, `beneficio_no_encontrado / servicio_no_encontrado`), que traduce a un mensaje legible para el usuario.

Check-in en iniciativas

El mecanismo es equivalente al de canje: el cliente obtiene la posición GPS del usuario mediante `Geolocator` y llama a la función RPC `checkin_iniciativa`, pasando el identificador de la iniciativa y las coordenadas. La validación de rango geográfico permitido, ventana horaria de verificación, coincidencia con el método de verificación configurado (QR) y ausencia de un check-in previo del mismo usuario ocurre en el servidor, evidenciada por los códigos de error tipados que la función devuelve (`fuera_de_horario`, `fuera_de_rango`, `sin_ubicacion_registrada`, entre otros). Adicionalmente, **CheckinScannerScreen** realiza una validación previa puramente del lado del cliente: compara el código escaneado contra el identificador de la iniciativa antes de siquiera invocar la función del servidor, evitando una llamada de red innecesaria ante un código evidentemente incorrecto.

Votación

A diferencia de los dos mecanismos anteriores, el registro de un voto —tanto en `iniciativas` como en `votaciones`— se realiza mediante un **insert directo** del cliente hacia las tablas `iniciativa_votes` y `votacion_votes` respectivamente, sin invocar ninguna función RPC intermedia. La prevención de votos duplicados no depende de una validación explícita de lógica de negocio en el servidor, sino de una **restricción de unicidad a nivel de base de datos**: el cliente captura el código de error `23505` (violación de restricción única de Postgres) devuelto por el intento de inserción duplicada, y lo traduce a la excepción tipada `AlreadyVotedException` / `AlreadyVotedIniciativaException`.

Distinción relevante para el diseño de flujos: el mecanismo de votación es más simple que el de canje o check-in —no requiere una función de negocio dedicada—, pero sigue siendo seguro

ante duplicados gracias a la restricción de unicidad de la base de datos, no a una verificación previa del cliente. Es un patrón válido para una operación de una sola regla (un voto por usuario por votación), pero no sería suficiente si el flujo de votación llegara a incorporar reglas adicionales (por ejemplo, ventanas de tiempo o límites de votos por categoría), que sí requerirían el mismo patrón de función RPC usado en canje y check-in.

Actualizaciones en tiempo real

La aplicación utiliza de forma extensiva el mecanismo de *streams* de `supabase_flutter` (`.stream(primaryKey: ['id'])`) sobre las tablas de mayor cambio frecuente: publicaciones del feed, iniciativas y sus votos, beneficios y servicios activos, notificaciones, balance de HeartCoin del usuario, historial de transacciones y comentarios de votación. Esto significa que, por ejemplo, un nuevo voto registrado por otro usuario se refleja en el Monitor de votaciones de Organización sin necesidad de que esa pantalla se recargue manualmente.

Problemas Detectados

1. **Inconsistencia en el nivel de validación server-side entre flujos transaccionales:** canje y check-in delegan su lógica de negocio completa a funciones RPC, mientras que votación depende únicamente de una restricción de unicidad de base de datos. Esto no representa una falla hoy —la única regla de negocio de votación (un voto por usuario) está correctamente protegida—, pero es una inconsistencia de patrón que podría volverse un problema si el flujo de votación incorpora reglas adicionales sin migrar al mismo mecanismo de RPC.
2. **Pantalla implementada sin punto de entrada de navegación:** `VotacionMonitorDetailScreen` existe completamente desarrollada (tasa de aprobación, participación, evolución de votos) pero no es alcanzable desde ningún flujo actual, representando trabajo de desarrollo ya invertido que no está generando valor visible para el usuario de Organización.
3. **Estado "Borrador" no utilizado por el flujo de creación:** la pantalla de listado de iniciativas de Organización ofrece un filtro por estado "Borrador", pero el asistente de creación no produce iniciativas en ese estado — toda iniciativa nueva entra directamente a votación, dejando el filtro sin utilidad práctica salvo que se edite manualmente el estado después de creada.
4. **Secciones "Próximamente" en el flujo de iniciativas de ahorro:** las pestañas y botones relacionados con seguimiento de proveedor y actividad reciente de una iniciativa de ahorro están presentes visualmente pero no tienen datos ni lógica detrás, pudiendo generar una expectativa de funcionalidad que el usuario no puede completar.
5. **Rutas de Empresa sin nombre registrado:** `CompanyServiciosScreen`, `CreateBeneficioScreen` y `CreateServicioScreen` solo son alcanzables por navegación directa, lo que dificulta, por ejemplo,

la implementación futura de enlaces profundos (*deep links*) o notificaciones push que requieran abrir esas pantallas directamente por nombre de ruta.

Riesgos e Impactos

- El riesgo del punto 1 es de **impacto en integridad de datos, actualmente latente**: no se materializa mientras la única regla de negocio de votación sea "un voto por usuario", pero se activaría en cuanto se agregue cualquier regla adicional que la restricción de unicidad de base de datos no pueda expresar por sí sola.
- El riesgo del punto 2 es de **impacto en aprovechamiento de la inversión de desarrollo**, sin riesgo de integridad ni de datos: es trabajo ya construido que simplemente no está expuesto al usuario.
- El riesgo del punto 3 es de **impacto en claridad del modelo de estados** para cualquier persona que dé mantenimiento futuro al flujo de creación de iniciativas, al no coincidir el estado inicial real con las opciones de filtro visibles en la interfaz.
- El riesgo del punto 4 es de **impacto en experiencia de usuario**, por expectativas no cumplidas ante funcionalidad visualmente presente pero inactiva.
- El riesgo del punto 5 es de **impacto en escalabilidad de la navegación**, materializable solo si el proyecto incorpora mecanismos que dependan de rutas nombradas (*deep links*, notificaciones push accionables).

Fortalezas Identificadas

- **Flujo de autenticación completo y consistente entre los tres roles**, con un único punto de entrada (Onboarding → Bienvenida → Registro/Login) y bloqueo efectivo de acceso hasta completar la verificación de correo.
- **Resolución automática de contexto en el escáner de canje**, reduciendo fricción para el usuario al no requerir que indique de antemano si está canjeando un beneficio o un servicio.
- **Correspondencia verificada entre generación y consumo de código QR**: el identificador mostrado en pantallas de QR de Organización y Empresa es exactamente el mismo que validan los escáneres del lado Personal, confirmando que el ciclo completo (generar código → escanearlo → aplicar la acción de negocio) está correctamente cerrado.
- **Delegación de lógica de negocio crítica al servidor** en los dos flujos transaccionales de mayor sensibilidad (canje de HeartCoin y check-in), evitando que la integridad del saldo de HeartCoin o del registro de asistencia dependa de que el cliente respete las reglas por sí solo.

- **Uso consistente de actualizaciones en tiempo real** en las pantallas donde la frescura del dato importa más (feed, votaciones, balance, notificaciones), sin necesidad de recarga manual por parte del usuario.

Áreas de Mejora

- Evaluar la migración del mecanismo de votación al mismo patrón de función RPC usado en canje y check-in, de forma preventiva, antes de que se introduzca una regla de negocio adicional que la restricción de unicidad de base de datos no pueda cubrir por sí sola.
- Definir si `VotacionMonitorDetailScreen` debe engancharse al flujo del Monitor de votaciones (por ejemplo, mediante un tap sobre la gráfica de barras) o retirarse del código si ya no corresponde a la dirección actual del producto.
- Revisar si el estado "Borrador" debe incorporarse como paso real del asistente de creación de iniciativas, o si el filtro correspondiente en la pantalla de listado debe ajustarse para reflejar que las iniciativas nuevas nunca inician en ese estado.
- Registrar rutas nombradas para `CompanyServiciosScreen`, `CreateBeneficioScreen` y `CreateServicioScreen`, anticipando necesidades futuras de navegación profunda.

Recomendaciones

1. Mantener el patrón de delegación a funciones RPC como estándar para cualquier flujo transaccional nuevo que involucre movimiento de HeartCoin o registro de participación, replicando el diseño ya validado en canje y check-in.
2. Decidir explícitamente, en el próximo ciclo de trabajo sobre el Monitor de votaciones, si se completa el enganche de `VotacionMonitorDetailScreen` o se retira, evitando que código funcional quede indefinidamente sin punto de entrada.
3. Documentar dentro del propio código (comentario en `CreateOrgIniciativaScreen`) la decisión de que las iniciativas nuevas inician siempre en estado "votación", para que futuras personas que trabajen en el asistente de creación no asuman erróneamente que "Borrador" es alcanzable desde ahí.
4. Priorizar el registro de rutas nombradas para las pantallas de Empresa señaladas antes de iniciar cualquier trabajo de notificaciones push accionables o enlaces profundos, dado que ese trabajo depende directamente de esta mejora.

Conclusión

El diseño de flujos de usuario de HeartCoin está completo y operativo de principio a fin para los tres roles del sistema. El flujo de entrada (onboarding, registro con verificación de correo, inicio de sesión y recuperación de contraseña) funciona de forma consistente independientemente del rol elegido, y cada rol cuenta con un recorrido de pantallas coherente hacia sus operaciones centrales: publicar y socializar contenido y votar iniciativas para Personal, crear y monitorear iniciativas para Organización, y publicar y gestionar beneficios y servicios para Empresa. Los flujos transaccionales de mayor relevancia —canje de HeartCoin y check-in por asistencia— están respaldados por funciones de servidor que garantizan la integridad de sus reglas de negocio independientemente del comportamiento del cliente, mientras que el flujo de votación, aunque funcional y protegido contra duplicados, sigue un patrón más simple que debe revisarse si el producto incorpora reglas de votación más complejas en el futuro. Las inconsistencias identificadas —una pantalla sin enganche de navegación, un estado de iniciativa no utilizado por su propio asistente de creación, y rutas sin nombre en el flujo de Empresa— son de naturaleza incremental y no comprometen la operatividad actual del sistema.